

COM4573

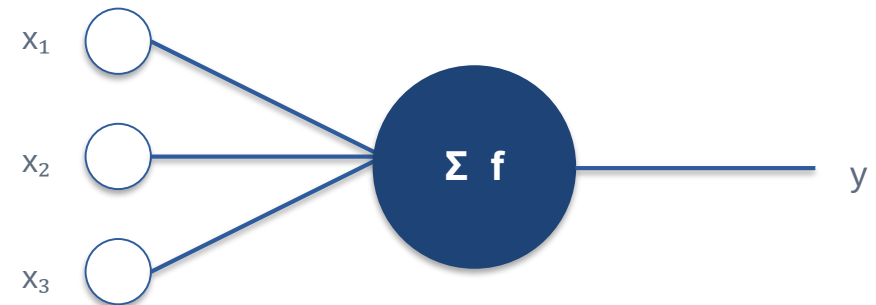
ARTIFICIAL NEURAL NETWORKS

Week 2 • Perceptron and Linear Classifiers – Python Implementation in COLAB

Fall Semester 2026-2027

Instructor: Dr. Hilal Akarkamçı

Faculty of Engineering • Department of Computer Engineering



Question of the week

Can one neuron learn to separate two classes?

Think about two groups of points on a plane. Could one straight line separate them?

This question connects the perceptron equation to geometry and learning.

From Week 1 to Week 2

1 • COMPUTE

$$z = \sum(w_i x_i) + b$$

A neuron combines inputs using weights and bias.

2 • ACTIVATE

$$y = f(z)$$

A step function can turn the score into a class decision.

3 • LEARN

If the prediction is wrong, adjust the parameters.

Today we make this update explicit.

Week 2 focus: decision boundary • linear separability • perceptron learning rule

What is a perceptron?

- A perceptron is a trainable linear binary classifier.
- It computes a weighted sum of the inputs, adds a bias, and applies a threshold.
- Its weights and bias determine a linear decision boundary.
- Learning changes the parameters when a training example is misclassified.

prediction = 1 if $w^T x + b \geq 0$, otherwise 0

The perceptron computation

Weighted sum

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Threshold decision

$$\hat{y} = 1 \text{ if } z \geq 0$$
$$\hat{y} = 0 \text{ if } z < 0$$

INTERPRETATION

Each weight controls how strongly an input influences the score.

The bias shifts the threshold.

The sign of z determines which side of the boundary the sample falls on.

From equation to decision boundary

For two inputs:

$$w_1x_1 + w_2x_2 + b = 0$$

This equation describes a straight line.
Points on opposite sides receive different class decisions.



How do weights and bias move the boundary?

WEIGHTS: ORIENTATION

Changing w changes the orientation of the decision boundary.

The relative values of w_1 and w_2 determine its slope.

BIAS: POSITION

Changing b shifts the boundary without changing the basic linear form.

The model learns both orientation and position.

Geometric idea: learning w and b means learning where the separating line should be.

Linear separability

LINEARLY SEPARABLE

There exists a straight line in 2D, a plane in 3D, or a hyperplane in higher dimensions that separates the classes.

A perceptron can learn such a boundary when the data are linearly separable.

NOT LINEARLY SEPARABLE

No single linear boundary can correctly separate every sample.

A single perceptron is then insufficient.

The limitation is important because it motivates multilayer neural networks.

Can more than one line separate the classes?

YES, SOMETIMES

For clearly separable data, several different straight lines may classify every training point correctly.

A perceptron needs a separating boundary, not a unique geometric line.

WHAT DOES TRAINING DO?

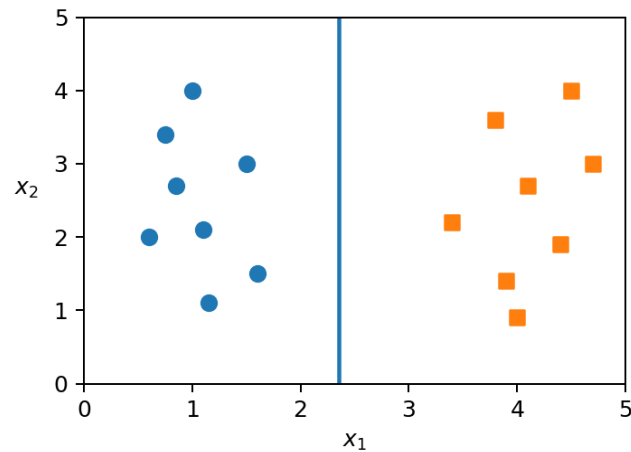
Training adjusts w and b according to the examples and update rule.

Different data order or settings can lead to different valid boundaries.

Class question: if two boundaries classify all training points correctly, how could we decide which one generalizes better?

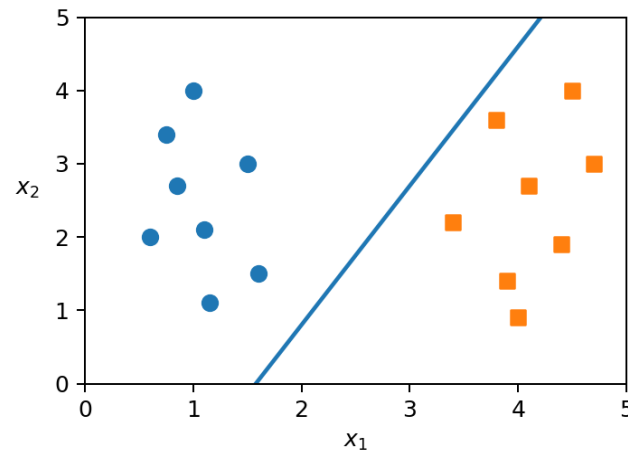
Examples of different separating boundaries

Example 1 • Vertical boundary



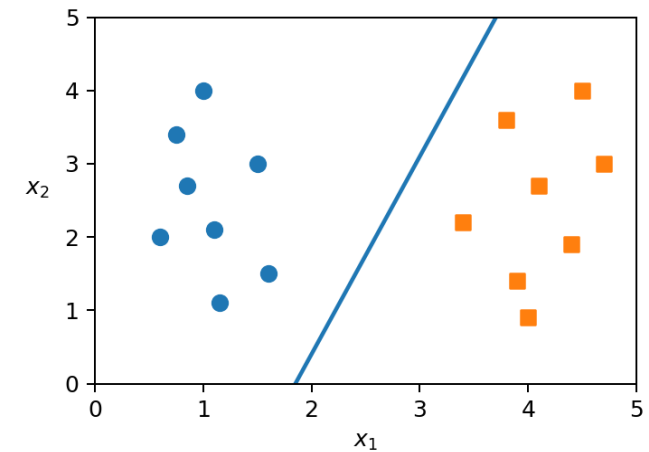
A vertical line separates the two classes correctly.

Example 2 • Slanted boundary



A slanted line also separates the two classes correctly.

Example 3 • Different valid boundary



Another straight line can separate the same classes correctly.

OBSERVATION

Several straight lines can classify the same training data correctly. The boundaries differ in orientation and margin.

Which decision boundary should we prefer?

1 • TEST UNSEEN DATA

If two boundaries classify the training samples correctly, training accuracy alone cannot tell us which will generalize better.

Compare performance on validation or other unseen examples.

2 • CONSIDER THE MARGIN

A boundary farther from the nearest training samples may be less sensitive to small changes.

Maximizing this margin is a central idea behind Support Vector Machines.

3 • WHAT DOES THE PERCEPTRON DO?

The classical perceptron does not search for the maximum-margin boundary.

Its final separating boundary can depend on the training examples, their order, initialization, and training settings.

KEY IDEA

Correctly separating the training data does not automatically identify the boundary that will generalize best.

Looking ahead: instead of finding any separating boundary, margin-based classifiers such as SVMs ask whether we can prefer a boundary with a larger margin.

AND as a linearly separable problem

Truth table

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

ONE POSSIBLE PERCEPTRON

$$w_1 = 1, w_2 = 1, b = -1.5$$

Only (1,1) gives a nonnegative score.

The four points can be separated by one line.

Question: what changes if we replace AND with OR?

XOR reveals the limitation

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

WHY ONE PERCEPTRON FAILS

The positive XOR cases lie on opposite corners.

No single straight line separates both positive points from both negative points.

This is not a training failure. It is a representational limitation of the model.

This is one reason hidden layers matter.

The perceptron learning rule

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$$

$$b \leftarrow b + \eta(y - \hat{y})$$

η • LEARNING RATE

Controls the size of each parameter update.

$y - \hat{y}$ • ERROR

Zero when the sample is classified correctly.

$\mathbf{x}$ • INPUT

The update is scaled by the current training example.

The rule changes parameters only when the current prediction is wrong.

What does the learning rate change?

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(\mathbf{y} - \hat{\mathbf{y}})\mathbf{x}$$

$$\mathbf{b} \leftarrow \mathbf{b} + \eta(\mathbf{y} - \hat{\mathbf{y}})$$

SMALL η

Smaller parameter changes. Learning may require more updates.

MODERATE η

Balanced updates can move the boundary efficiently toward a useful separator.

LARGE η

Large parameter changes can make the boundary move aggressively from one error to the next.

The learning rate controls update size. It does not change the fact that a single perceptron is a linear model.

One update by hand

Suppose:

$$x = [1, 2]$$

$$y = 1$$

$$w = [0.2, -0.1], \quad b = -0.2$$

$$\eta = 0.1$$

$$z = 0.2(1) - 0.1(2) - 0.2 = -0.2$$

Therefore $\hat{y} = 0$.

UPDATE

$$\text{error} = y - \hat{y} = 1$$

$$\begin{aligned} w_{\text{new}} &= [0.2, -0.1] + 0.1(1)[1, 2] \\ &= [0.3, 0.1] \end{aligned}$$

$$b_{\text{new}} = -0.2 + 0.1(1) = -0.1$$

The update moves the boundary in a direction that favors the correct class.

What is an epoch?

SAMPLE

One training example is presented to the model.

UPDATE

The model may change its parameters after an error.

EPOCH

One complete pass through all training examples.

Training can repeat for multiple epochs until the model converges or a stopping rule is reached.

When does perceptron training converge?

LINEARLY SEPARABLE

For linearly separable training data, the classic perceptron learning procedure can reach a separating solution after a finite number of updates.

NOT SEPARABLE

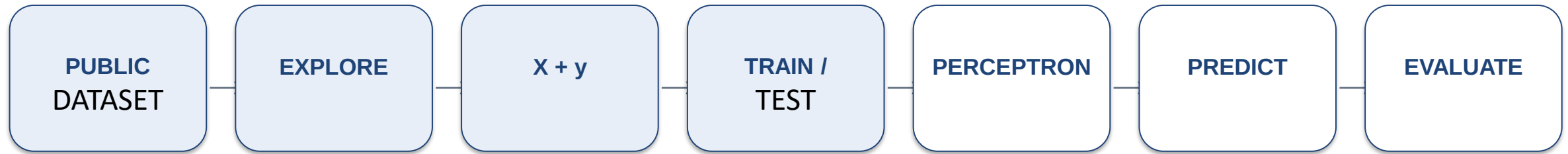
If no perfect linear separator exists, the algorithm cannot achieve zero classification error on every training sample.

PRACTICAL STOPPING

Implementations use limits such as a maximum number of iterations or a tolerance-based stopping rule.

Important distinction: failure to reach perfect separation may reflect model capacity, not simply insufficient training.

From equations to a public dataset



Live Colab question: where are the inputs, and where is the answer the model should learn?

We will use the public Iris dataset for a small, transparent classification example.

Iris dataset: what are we looking at?

150 SAMPLES

Each row describes one iris flower.

4 FEATURES

Sepal length, sepal width, petal length, petal width.

3 CLASSES

Setosa, Versicolor, Virginica.

For the perceptron demonstration, we first use two classes and two features so the boundary can be visualized.

Why start with two classes and two features?

VISUALIZE

Two features let us draw every sample on a 2D plane and see the learned boundary directly.

SIMPLIFY

Two classes keep the first perceptron example focused on binary classification and linear separability.

THEN EXTEND

The full Iris dataset has four features and three classes. We return to the richer problem when introducing a small ANN.

Teaching strategy: simplify the geometry first, then increase the model and data complexity.

Colab 1 • Load and inspect

```
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris(as_frame=True)
df = iris.frame

df.head()
```

ASK THE CLASS

What does one row represent?

Which columns are inputs?

Which column is the target?

What type of learning problem is this?

Reading the Iris dataset • X and y

Sepal length	Sepal width	Petal length	Petal width	Target	Class name
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Class name is shown only to make the numeric target code readable; it is not an additional model input.

For the first perceptron demo, we later select two features and two classes so the decision boundary can be visualized.

READ THE DATASET

- X = input features: sepal length, sepal width, petal length, petal width.
- y = target (species): 0 = setosa, 1 = versicolor, 2 = virginica.
- Problem type: supervised multiclass classification.
- Class names explain the target codes; they are not additional input features.

Colab 2 • Visualize before training

```
subset = df[df['target'] < 2]

plt.scatter(
    subset['sepal length (cm)'],
    subset['sepal width (cm)'],
    c=subset['target']
)
plt.xlabel('Sepal length (cm)')
plt.ylabel('Sepal width (cm)')
plt.show()
```

PAUSE BEFORE TRAINING

Can you visually separate the two classes?

Where would you draw the line?

Which points look difficult?

Colab 3 • Train the perceptron

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
```

```
X = subset[['sepal length (cm)', 'sepal width (cm)']]
y = subset['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

```
model = Perceptron(random_state=42)
model.fit(X_train_s, y_train)
```

CONNECT TO THEORY

fit() estimates the weights and bias.

The model still represents one linear decision boundary.

Scaling is fitted on the training set only.

Why standardize the input features?

COMMON SCALE

StandardScaler centers each training feature and scales it using its training-set standard deviation.

This puts features on comparable numerical scales.

AVOID LEAKAGE

`fit_transform()` is applied only to `X_train`.

The already fitted scaler is then used to transform `X_test`.

Rule to remember: learn preprocessing parameters from the training data, then apply the same transformation to unseen data.

What did the perceptron learn?

Inspect the learned parameters

```
print(model.coef_)  
print(model.intercept_)
```

INTERPRETATION

coef_ contains the learned feature weights.

intercept_ contains the learned bias.

Together they define the line that separates the two predicted classes.

The symbols w and b from the equation are now actual learned numbers.

Evaluate on unseen examples

```
from sklearn.metrics import accuracy_score,  
confusion_matrix  
  
y_pred = model.predict(X_test_s)  
  
print('Accuracy:', accuracy_score(y_test, y_pred))  
print('Confusion matrix:')  
print(confusion_matrix(y_test, y_pred))
```

WHY TEST DATA?

Training accuracy asks how well the model fits examples it saw.

Test accuracy asks how well it predicts examples that were held out.

Do not interpret one small test result as a universal performance guarantee.

Draw the learned decision boundary

The most important visual connection:

weights + bias $\rightarrow$ a line in feature space.

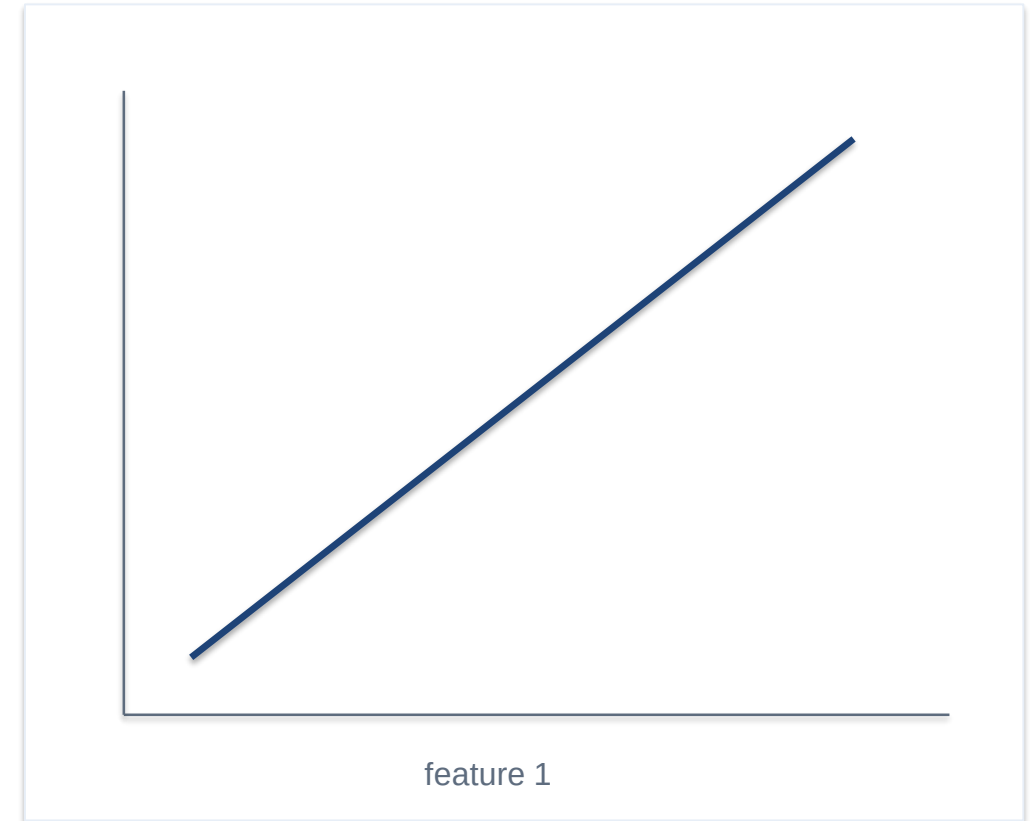
CLASS DISCUSSION

Which side is predicted as class 0?

Which side is class 1?

Do any test samples fall on the wrong side?

f
e
a
t
u
r
e
2



Look beyond one accuracy number

CONFUSION MATRIX

Shows which true classes were predicted correctly and where mistakes occurred.

It tells us more than a single summary score.

ERROR INSPECTION

Ask which samples are misclassified and where they lie relative to the learned boundary.

Errors near overlap are especially informative.

A useful evaluation question is not only “How many were correct?” but also “Which examples were wrong, and why?”

From a perceptron to a small ANN

One perceptron

inputs $\rightarrow$ weighted sum $\rightarrow$ threshold $\rightarrow$
class

A small multilayer network

inputs $\rightarrow$ hidden neurons $\rightarrow$ output
neurons $\rightarrow$ class

Adding hidden units allows the model to build nonlinear decision functions.

Week 3 will open this box: multilayer perceptrons.

Optional preview • A tiny ANN on all 3 classes

```
import tensorflow as tf

ann = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

ann.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

ann.fit(X_train, y_train, epochs=50, verbose=0)
```

PREVIEW, NOT FULL THEORY

The network has a hidden layer with 8 neurons.

ReLU introduces nonlinearity.

Softmax produces three class scores.

We will study these components in later weeks.

Perceptron vs small MLP

PERCEPTRON

Single trainable linear decision unit

Binary decision in the classic form

Linear boundary

Transparent connection between parameters and geometry

MULTILAYER PERCEPTRON

Multiple neurons organized into layers

Can support multiclass outputs

Nonlinear boundaries with nonlinear activations

More expressive, but training is more involved

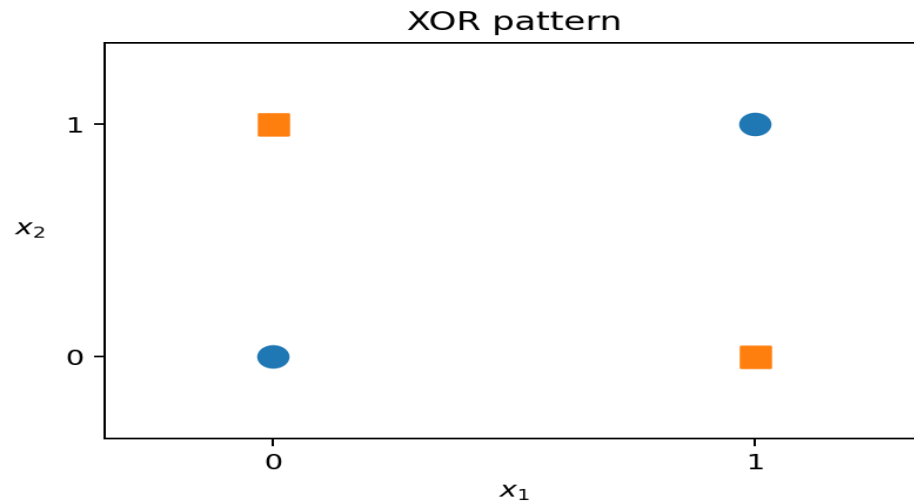
Five-minute challenge • Design a nonlinear classification problem

1. Sketch two classes that cannot be separated by one straight line.
2. Explain why a single perceptron would fail on your sketch.
3. Suggest how two or more linear boundaries could help.
4. Connect your idea to XOR, concentric regions, or another simple pattern.
5. Be ready to explain your reasoning in one minute.

Goal: move from “a perceptron cannot solve it” to “what additional representation might we need?”

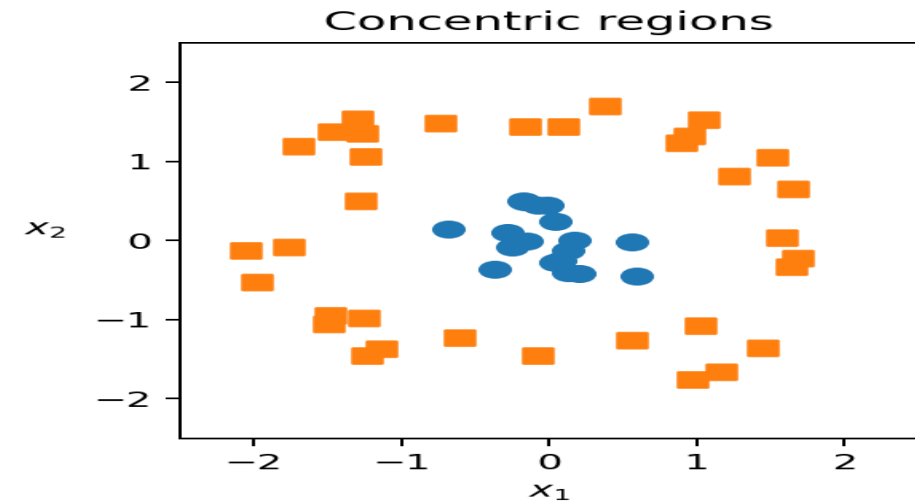
Examples • Not linearly separable

Example 1 • XOR



The two classes occupy opposite corners. No single straight line can separate all Class 0 points from all Class 1 points.

Example 2 • Concentric regions



One class is surrounded by the other. A single straight line creates only two half-spaces, so it cannot form an enclosed decision region.

COMMON IDEA

The problem is not insufficient training. The required decision boundary is nonlinear. What could several neurons do that one perceptron cannot?

Back to the question of the week

Can one neuron learn to separate two classes?

Yes, when a suitable linear decision boundary exists and the learning process finds parameters that represent it.

But a single perceptron cannot represent every classification pattern. XOR is the classic counterexample.

Week 2 • Key takeaways

- A perceptron is a trainable linear classifier.
- Weights and bias define a decision boundary.
- Linear separability determines whether one perceptron can perfectly separate the classes.
- The perceptron learning rule updates parameters after classification errors.
- AND and OR are linearly separable; XOR is not.
- A real dataset makes the link between equations, learned parameters, predictions, and evaluation visible.

Questions 1-5

- Write the perceptron decision rule and explain the roles of weights and bias.
- What does it mean for two classes to be linearly separable?
- For AND, propose weights and a bias that produce the correct outputs.
- Why can a single perceptron not represent XOR?
- Explain the purpose of the learning rate in the perceptron update rule.

Questions 6-10

- In the Iris demo, identify X , y , the training set, and the test set.
- Why should a scaler be fitted on the training data rather than on all data?
- What information is stored in `model.coef_` and `model.intercept_`?
- Why is a confusion matrix useful in addition to accuracy?
- Describe one limitation of the classroom Iris demonstration and one way you would extend it.

Before Week 3

REVIEW

Perceptron equation
Decision boundary
Linear separability
Learning rule

THINK

How could several linear units work together to solve a nonlinear problem?

PREVIEW

Multilayer perceptrons
Hidden layers
Nonlinear activations
Forward propagation

Week 3 • Multilayer Perceptrons

Colab output • What does the dataset contain?

Sepal length	Sepal width	Petal length	Petal width	Target	Class name
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

The scikit-learn Iris frame contains 150 samples, 4 predictive features, and the target column.

READ THE OUTPUT

- Each row is one flower observation.
- The four measurement columns are input features.
- target is the supplied class label: 0, 1, or 2.
- The numeric target codes are categories, not measured quantities.

Colab output • Visualize before training

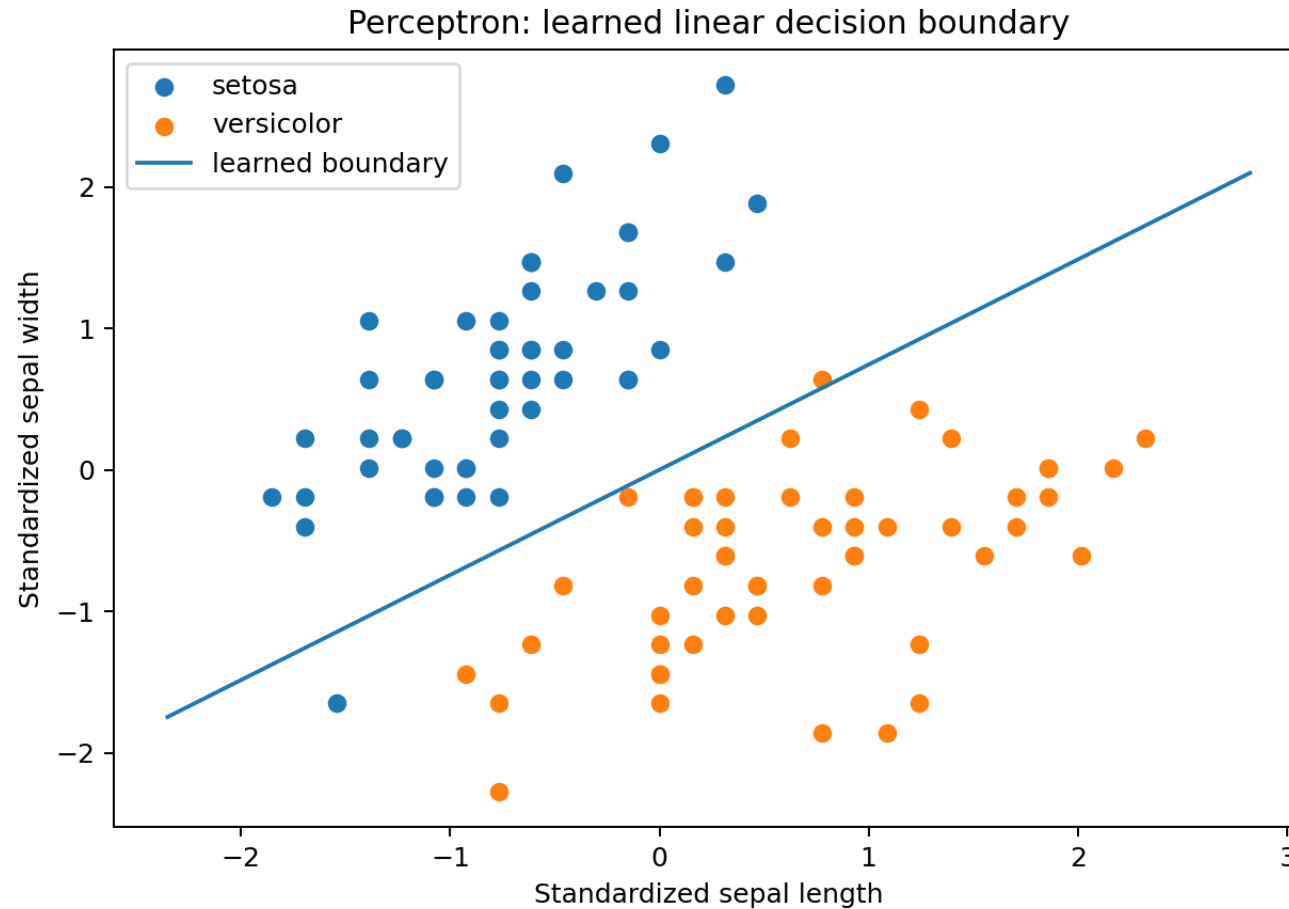


PAUSE AND ASK

- Can one straight line separate these two classes?
- Where would you draw the boundary?
- Why might some points near the overlap be difficult?

We deliberately use two classes and two features here so that the geometry of the perceptron can be seen directly.

Colab output • The learned decision boundary

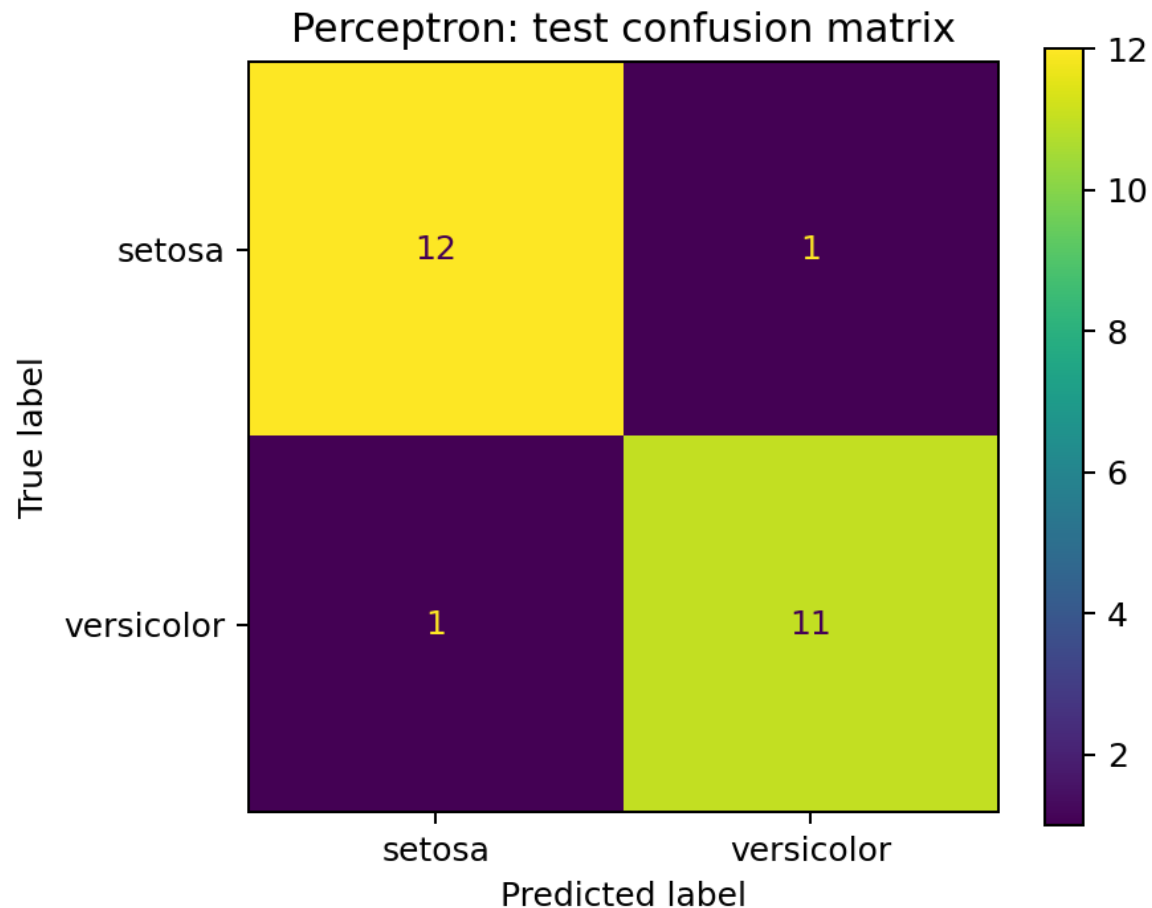


CONNECT BACK TO THE EQUATION

- Learned weights: 1.546, -2.081
- Learned bias: 0.000
- w and b determine the position and orientation of the line.
- The boundary is linear because the perceptron uses $w^T x + b$.

The symbols from the equation are now learned numerical parameters.

Colab output • Evaluate unseen examples



WHAT DOES THIS TELL US?

- Test accuracy in this fixed split: 0.92
- The confusion matrix shows correct and incorrect predictions by class.
- A test result evaluates held-out examples, not the training examples.
- One small split is a teaching example, not a universal performance claim.

Next question: what happens when one linear boundary is not expressive enough?

Questions to discuss before revealing the answers

1. What did the perceptron learn?
2. Where can we see w and b in the trained model?
3. Why was the decision boundary linear?
4. What limitation did XOR reveal?
5. What changed when we introduced a hidden layer?

Use the Colab outputs and the XOR example as evidence for your answers.

Suggested answers • Perceptron and linearity

1 • WHAT DID IT LEARN?

The perceptron learned weights w and a bias b that define its decision rule.

2 • WHERE ARE w AND b ?

In scikit-learn, `model.coef_` stores the learned weights and `model.intercept_` stores the learned bias.

3 • WHY A LINEAR BOUNDARY?

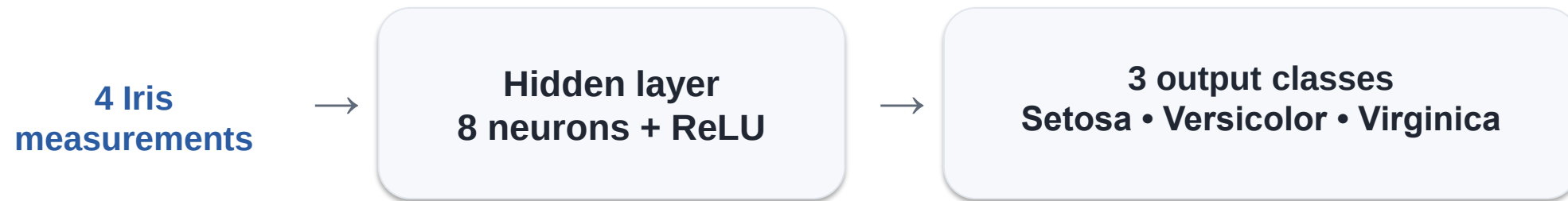
The decision boundary satisfies $w_1x_1 + w_2x_2 + b = 0$. With two features, this is a straight line.

4 • WHAT DID XOR REVEAL?

XOR is not linearly separable. No single straight line can separate its two classes, so one perceptron is insufficient.

What changes when we introduce a hidden layer?

A hidden layer with nonlinear activation functions allows the network to combine learned transformations and represent nonlinear relationships.



FROM SIMPLE TO COMPLEX

- A single perceptron learns one linear boundary.
- A hidden layer plus nonlinearity can form more complex decision regions.
- XOR is the classic minimal example.
- A realistic extension is classifying all three Iris species from all four measurements.

References and demo sources

- Aggarwal, C. C. (2023). Neural Networks and Deep Learning: A Textbook (2nd ed.). Springer. DOI: 10.1007/978-3-031-29642-0
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Official online edition: deeplearningbook.org
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. DOI: 10.1037/h0042519
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179-188. DOI: 10.1111/j.1469-1809.1936.tb02137.x
- Scikit-learn. `load_iris` documentation and User Guide: Iris plants dataset. scikit-learn.org/stable/modules/generated/sklearn.datasets.load_iris.html
- Scikit-learn documentation: `Perceptron`, `train_test_split`, `StandardScaler`, `accuracy_score`, and `confusion_matrix`.
- TensorFlow / Keras documentation: `Sequential` model, `Dense` layers, `ReLU`, `softmax`, and `sparse categorical cross-entropy`.